LAB ASSIGNMENT-6

Submitted by:

Abhinav Khandelwal

1024150194
2023

1. Implement a class Library that contains an array of Book objects. The Book class should have attributes like title, author, and ISBN. The Library class should provide following methods:
    a. bool addNewBook(title, author, ISBN)
    b. bool removeBooks(ISBN)
    c. displayDetails() to list the books.

    Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

```cpp
#include <iostream>
#include <string>
using namespace std;
class Book {
public:
    string title;
    string author;
    string ISBN;
};
class Library {
public:
    Book b[10];
    int count;
    Library() {
        count = 0;
    }
    bool addNewBook(string &title, string &author, string &ISBN) {
        b[count].title = title;
        b[count].author = author;
        b[count].ISBN = ISBN;
        count++;
        return true;
    }
```

```cpp
    bool removeBooks(string &ISBN);
    void displayDetails() {
        cout << "\nBooks List:\n";
        for (int i = 0; i < count; i++) {
            cout << "Title: " << b[i].title << endl;
            cout << "Author: " << b[i].author << endl;
            cout << "ISBN: " << b[i].ISBN << endl;
            cout << "------------\n";
        }
    }
};bool Library::removeBooks(string &ISBN) {
    for (int i = 0; i < count; i++) {
        if (b[i].ISBN == ISBN) {
            for (int j = i; j < count - 1; j++) {
                b[j] = b[j + 1];
            }
            count--;
            return true;
        }
    }
    return false;
}int main() {
    Library l;
    string t, a, i;
    t = "C++"; a = "Bjarne"; i = "101";
    l.addNewBook(t, a, i);
    t = "DSA"; a = "Mark"; i = "102";
    l.addNewBook(t, a, i);
    t = "OOP"; a = "James"; i = "103";
    l.addNewBook(t, a, i);
    t = "Algo"; a = "CLRS"; i = "104";
    l.addNewBook(t, a, i);
    t = "Math"; a = "RD"; i = "105";
    l.addNewBook(t, a, i);
    l.displayDetails();
    string rem = "103";
    l.removeBooks(rem);
    cout << "\nAfter Deletion:\n";
    l.displayDetails();
    return 0;
}
```

OUTPUT:

```
C:\Users\pg664\OneDrive\De
ISBN: 104
------------
Title: Math
Author: RD
ISBN: 105
------------

After Deletion:

Books List:
Title: C++
Author: Bjarne
ISBN: 101
------------
Title: DSA
Author: Mark
ISBN: 102
------------
Title: Algo
Author: CLRS
ISBN: 104
------------
Title: Math
Author: RD
ISBN: 105
------------
```

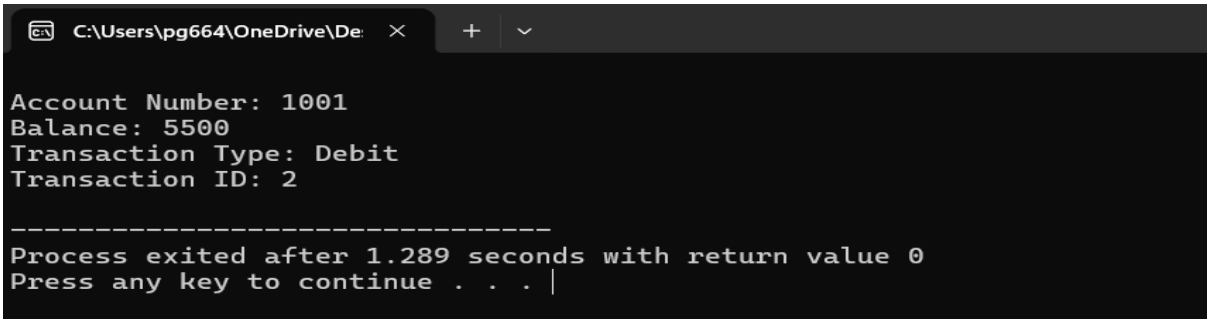
2. Book Class Implementation

Implement the question 1 using constructor. Do the following modification:

- a. Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., title, author, and ISBN.
- b. Use the "this" pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.
- c. Create function bool removeBooks(ISBN) same as the question 1.
- d. displayDetails() same as question 1.
- e. Create array of objects using 1. Initializer list, 2. Dynamic initialization.

```cpp
#include <iostream>
using namespace std;
class Account {
public:
    const long account_number;
    long transaction_id;
    string transaction_type;
    double balance;
    Account(long acc, double bal) : account_number(acc) {
        balance = bal;
        transaction_id = 0;
        transaction_type = "None";
    }
    long depositAmount(const long to, const long from, const double amount) {
        balance = balance + amount;
        transaction_type = "Credit";
        transaction_id++;
        return transaction_id;
    }
    long creditAmount(const long to, const long from, const double amount) {
        if (balance >= amount) {
            balance = balance - amount;
            transaction_type = "Debit";
            transaction_id++;
        }
        return transaction_id;
    }
    void displayDetails() const {
        cout << "\nAccount Number: " << account_number << endl;
        cout << "Balance: " << balance << endl;
        cout << "Transaction Type: " << transaction_type << endl;
        cout << "Transaction ID: " << transaction_id << endl;
    }
};
int main() {
    Account a1(1001, 5000);
    Account a2(1002, 3000);
    Account a3(1003, 7000);
    Account a4(1004, 2000);
    Account a5(1005, 9000);
    a1.depositAmount(1001, 1002, 1000);
    a1.creditAmount(1001, 1003, 500);
    a1.displayDetails();
    return 0;
}
```

OUTPUT:



3. Account Class Implementation

Create a class Account with attributes account number (*of const long type*), transaction ID (*of long type*), transaction type (credit/debit) (*of string type*), and balance (*of double type*). The Account class should provide following methods:

- a. long depositAmount(to, from, amount). The "to" and "from" are the account number of the sender and receiver. The function should return the transaction id.
- b. long creditAmount(to, from, amount) similar to the depositAmount function. This function should also return the transaction id.
- c. void displayDetails() should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.
- d. In all the functions, make all the arguments as the const arguments. Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit

```
#include <iostream>
using namespace std;
class Account {
private:
    long account_number;
    long transaction_id;
    string transaction_type;
    double balance;
public:
    Account(long account_number, double balance) {
        this->account_number = account_number;
        this->balance = balance;
        this->transaction_id = 0;
        this->transaction_type = "None";
    }
```

```
    long depositAmount(long to, long from, double amount) {
        this->balance = this->balance + amount;
        this->transaction_type = "Credit";
        this->transaction_id++;

        return this->transaction_id;
    }
    long creditAmount(long to, long from, double amount) {
        if (this->balance >= amount) {
            this->balance = this->balance - amount;
            this->transaction_type = "Debit";
            this->transaction_id++;
        }
        return this->transaction_id;
    }
    void displayDetails() {
        cout << "\nAccount No: " << this->account_number << endl;
        cout << "Balance: " << this->balance << endl;
        cout << "Last Transaction: " << this->transaction_type << endl;
        cout << "Transaction ID: " << this->transaction_id << endl;
    }
};
int main() {
    Account a1(1001, 5000);

    a1.depositAmount(1001, 1002, 1000);
    a1.creditAmount(1001, 1003, 500);

    a1.displayDetails();

    return 0;
}
```

OUTPUT:

```
C:\Users\pg664\OneDrive\De

Account No: 1001
Balance: 5500
Last Transaction: Debit
Transaction ID: 2

--------------------------------
Process exited after 1.708 seconds with return value 0
Press any key to continue . . .
```

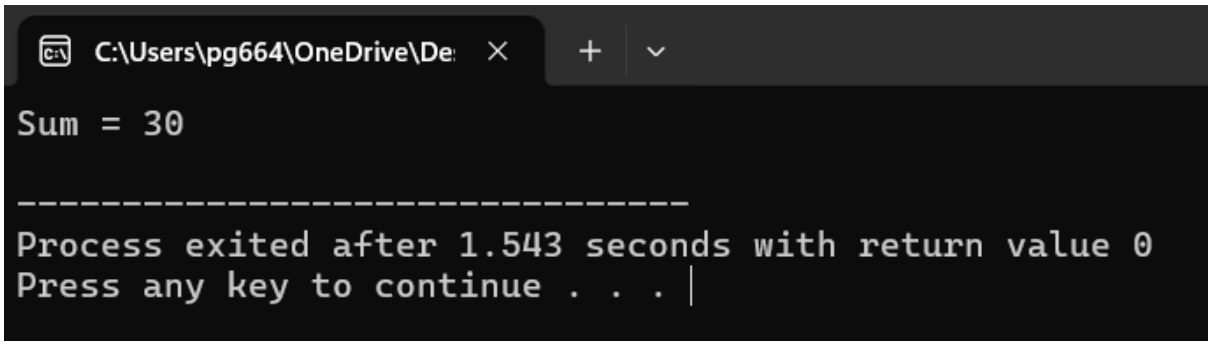
4. Friend Functions

Write a program to add data objects of two different classes using friend functions

```
#include <iostream>
using namespace std;
class B;
class A {
private:
    int num1;
public:
    A() {
        num1 = 10;
    }
    friend void add(A, B);
};
class B {
private:
    int num2;
public:
    B() {
        num2 = 20;
    }
    friend void add(A, B);
```

```
21  };
22  void add(A a, B b) {
23      int sum = a.num1 + b.num2;
24      cout << "Sum = " << sum << endl;
25  }
26  int main() {
27      A obj1;
28      B obj2;
29      add(obj1, obj2);
30      return 0;
31  }
```

OUTPUT:



5. Complex Number Class

Define a class named Complex with properties (real and imaginary) and methods as per following details.

- a. Parameterized constructor and copy constructor to initialize object values.
- b. void display () to display complex number.
- c. void sum (Complex, Complex) to add two complex numbers (objects of Complex class). Implement this function as the friend function

```
1   #include <iostream>
2   using namespace std;
3   class Complex {
4   private:
5       int real;
6       int imag;
7   public:
8       Complex(int r, int i) {
9           real = r;
10          imag = i;
11      }
12      Complex(const Complex &c) {
13          real = c.real;
14          imag = c.imag;
15      }
16      void display() {
17          cout << real << " + " << imag << "i" << endl;
18      }
19      friend void sum(Complex, Complex);
20  };
21  void sum(Complex c1, Complex c2) {
22      int r = c1.real + c2.real;
23      int i = c1.imag + c2.imag;
24      cout << "Sum = " << r << " + " << i << "i" << endl;
25  }
```

```
25  }
26  int main() {
27      Complex c1(2, 3);
28      Complex c2(4, 5);
29      Complex c3 = c1;
30      cout << "First: ";
31      c1.display();
32      cout << "Second: ";
33      c2.display();
34
35      sum(c1, c2);
36
37      return 0;
38  }
```

OUTPUT: